Project Assignment 3: Interrogating the Data

0 OF 1 QUESTIONS REMAINING

Form Content

The scenario

You are part of the Trust and Data team for your platform. Other teams use your data to make important decisions, so your team is responsible for making sure the data and queries are correct. Two weeks ago, another team created a dashboard reporting that about half of all events end in a failure state. Based on this result, leadership froze a quarterly payout. Someone familiar with the platform questions the result. Their experience does not match the failure rate shown on the dashboard. There is another concern: no one has checked the SQL query behind the dashboard. The query runs without an error, and its results looked reasonable, so everyone assumed it was correct.

Your job

In this assignment, you will:

1. Explore the provided data.
2. Determine what the data actually contains.
3. Reproduce and fix two SQL problems.
4. Write a corrected query in three different ways.
5. Explain your findings to a non-technical audience.

Task 3.1: Establish what the data contains

Before you can audit a rate you need to know the population it was computed over. Write a set of queries that answer these questions, and record what each one told you.

The questions

1. Find active, highly rated producers. Use your theme's filter condition from the appendix. Sort them and return the first ten.
2. Find all possible event statuses. Display the distinct values in the event status column. Look carefully at the results. Are there more statuses than you expected?
3. Filter the events in two different ways. Write: one query that filters a numeric range, and one query that filters using an explicit list of values with IN.
4. Work with text and missing values. Write queries that: find producers whose names match a text pattern using LIKE, and identify records where a reason/value is missing (NULL). Use COALESCE to replace a missing value with a useful label such as: No reason provided.
5. Make the results easier to understand. Create a query with calculated/computed columns that would make sense to someone who does not work with SQL. Depending on your theme, you might calculate or label:
   - a payout amount,
   - a time of day,
   - a value/rating category,
   - or another useful business measure.

   Use CASE where appropriate to create plain-language categories.

```sql
-- 3.1.1 Find active, highly rated drivers. Sort and return the first ten.
SELECT driver_id, driver_name, avg_rating  337 ms
FROM  drivers
WHERE is_active = TRUE
AND avg_rating >= 4.5
ORDER BY avg_rating DESC
LIMIT 10;
```

Output | ex603_data.public.drivers

Tx: Auto | DDL

	driver_id	driver_name	avg_rating
1	1	Driver_01	4.97
2	2	Driver_02	4.94
3	13	Driver_13	4.94
4	4	Driver_04	4.88
5	5	Driver_05	4.85

```sql
-- 3.1.2 - All possible event statuses. Display the distinct values in the trip status column.
SELECT DISTINCT trip_status  336 ms
FROM  trips;
```

Output | ex603_data.public.trips

Tx: Auto | DDL

	trip_status
1	disputed
2	completed
3	cancelled

```sql
-- 3.1.3 Filter the trips in two different ways. First way: one query that filters a numeric range
SELECT *  342 ms
FROM  trips
WHERE distance_km < 10;
```

Output | 3.1.3 Filter the tri...lters a numeric range

Tx: Auto | DDL | CSV

	trip_id	rider_id	driver_id	requested_at	completed_at	pickup_city	fare_amount	distance_km	surge_multi…
1	2	28	1	2023-06-09 08:54:00.000000	2023-06-09 09:37:00.000000	Danang	7.40	3.12	
2	5	25	1	2023-06-27 21:17:00.000000	2023-06-27 21:43:00.000000	Hanoi	7.07	3.93	
3	8	4	1	2023-01-16 21:46:00.000000	<null>	Hue	0.00	2.55	
4	12	19	2	2023-01-20 08:07:00.000000	<null>	Hue	0.00	5.54	
5	14	16	2	2023-01-27 17:06:00.000000	<null>	Hanoi	0.00	9.12	
6	15	27	2	2023-11-05 23:52:00.000000	2023-11-06 00:14:00.000000	Hanoi	20.29	8.05	
7	19	30	2	2023-01-20 11:49:00.000000	2023-01-20 12:37:00.000000	Saigon	15.21	8.21	
8	20	6	2	2023-04-26 01:39:00.000000	2023-04-26 02:22:00.000000	Danang	13.50	8.27	
9	29	1	3	2023-07-06 19:25:00.000000	2023-07-06 20:19:00.000000	Hanoi	11.15	4.94	
10	31	21	3	2023-09-04 12:21:00.000000	<null>	Hue	0.00	4.84	
11	32	13	3	2023-12-28 02:28:00.000000	2023-12-28 02:39:00.000000	Danang	7.86	4.71	
12	37	6	3	2023-05-12 06:59:00.000000	2023-05-12 07:48:00.000000	Hue	13.24	9.01	
13	39	7	3	2023-06-02 18:07:00.000000	2023-06-02 18:39:00.000000	Danang	7.46	4.08	
14	44	16	4	2023-09-05 08:58:00.000000	<null>	Danang	0.00	5.40	
15	45	11	4	2023-05-19 01:12:00.000000	2023-05-19 01:37:00.000000	Saigon	8.95	5.28	
16	50	4	5	2023-11-26 01:07:00.000000	2023-11-26 02:01:00.000000	Danang	10.26	9.31	
17	52	23	5	2023-10-11 05:57:00.000000	<null>	Saigon	0.00	7.84	
18	64	6	5	2023-06-02 17:01:00.000000	2023-06-02 17:39:00.000000	Hue	16.57	6.48	
19	71	16	6	2023-12-05 08:29:00.000000	2023-12-05 09:14:00.000000	Saigon	6.17	3.98	
20	76	2	6	2023-10-06 05:08:00.000000	2023-10-06 05:41:00.000000	Danang	5.24	3.48	
21	77	1	6	2023-03-18 23:27:00.000000	2023-03-18 23:47:00.000000	Haiphong	8.88	3.02	
22	79	23	6	2023-07-08 15:16:00.000000	2023-07-08 15:37:00.000000	Hanoi	4.65	2.01	
23	82	23	6	2023-01-13 17:20:00.000000	2023-01-13 18:10:00.000000	Saigon	18.77	9.31	
24	85	26	6	2023-10-21 20:43:00.000000	[illegible]:26:00.000000	Danang	16.46	6.83	

65 rows

```sql
-- 3.1.3 Filter the trips in two different ways. Second way: one query that filters using an explicit list of values with IN.
SELECT *  379 ms
FROM  trips
WHERE pickup_city IN ('Hue', 'Hanoi');
```

Output | ex603_data.public.trips

Tx: Auto | DDL | CSV

	trip_id	rider_id	driver_id	requested_at	completed_at	pickup_city	fare_amount	distance_km	surge_multi
1	4	1	1	2023-01-01 15:48:00.000000	<null>	Hue	0.00	10.65	
2	5	25	1	2023-06-27 21:17:00.000000	2023-06-27 21:43:00.000000	Hanoi	7.07	3.93	
3	8	4	1	2023-01-16 21:46:00.000000	<null>	Hue	0.00	2.55	
4	9	15	1	2023-03-26 10:22:00.000000	2023-03-26 11:02:00.000000	Hue	20.06	19.69	
5	12	19	2	2023-01-20 08:07:00.000000	<null>	Hue	0.00	5.54	
6	13	22	2	2023-04-12 22:27:00.000000	2023-04-12 22:53:00.000000	Hanoi	19.89	11.98	
7	14	16	2	2023-01-27 17:06:00.000000	<null>	Hanoi	0.00	9.12	
8	15	27	2	2023-11-05 23:52:00.000000	2023-11-06 00:14:00.000000	Hanoi	20.29	8.05	
9	16	23	2	2023-05-01 10:38:00.000000	2023-05-01 11:10:00.000000	Hanoi	19.83	12.94	
10	17	11	2	2023-08-11 08:48:00.000000	2023-08-11 09:29:00.000000	Hue	29.42	16.03	
11	27	15	2	2023-07-25 22:44:00.000000	2023-07-25 22:52:00.000000	Hanoi	24.02	15.54	
12	29	1	3	2023-07-06 19:25:00.000000	2023-07-06 20:19:00.000000	Hanoi	11.15	4.94	
13	31	21	3	2023-09-04 12:21:00.000000	<null>	Hue	0.00	4.84	
14	34	11	3	2023-05-12 17:06:00.000000	2023-05-12 17:41:00.000000	Hanoi	21.60	21.88	
15	35	8	3	2023-02-28 17:00:00.000000	<null>	Hue	0.00	19.42	
16	37	6	3	2023-05-12 06:59:00.000000	2023-05-12 07:48:00.000000	Hue	13.24	9.01	
17	40	6	3	2023-05-03 19:33:00.000000	<null>	Hanoi	0.00	19.81	
18	42	12	4	2023-08-17 02:06:00.000000	2023-08-17 02:34:00.000000	Hanoi	14.34	12.03	
19	43	19	4	2023-10-15 19:01:00.000000	2023-10-15 19:34:00.000000	Hue	70.86	26.11	
20	49	26	5	2023-05-05 17:19:00.000000	2023-05-05 17:59:00.000000	Hanoi	32.56	20.13	
21	51	29	5	2023-05-12 19:00:00.000000	2023-05-12 19:51:00.000000	Hue	20.17	14.64	
22	54	13	5	2023-10-11 06:21:00.000000	2023-10-11 06:36:00.000000	Hue	37.59	24.32	
23	60	9	5	2023-12-20 19:28:00.000000	2023-12-20 19:55:00.000000	Hue	48.41	22.51	
24	62	28	5	2023-07-14 04:15:00.000000	2023-07-14 04:50:00.000000	Hanoi	23.90	22.79	

78 rows

```sql
-- 3.1.4 find producers (drivers) whose names match a text pattern using LIKE, and identify records where a reason/value is missing (NULL). Use COALESCE to replace a missing value with a useful label
SELECT  354 ms
    d.driver_id,
    d.driver_name,
    t.trip_id,
    completed_at,
    coalesce(t.cancellation_reason, 'Not Cancelled') AS cancellation_reason
FROM drivers d
join trips t
    1<->1..n: ON d.driver_id=t.driver_id
WHERE d.driver_name LIKE 'Driver_1%'
AND t.cancellation_reason IS NULL;
```

s > Database > postgres@localhost > q3_1.sql

Output | Result 73

CSV

driver_id	driver_name	trip_id	completed_at	cancellation_reason
10	Driver_10	138	2023-09-25 08:24:00.000000	Not Cancelled
10	Driver_10	139	2023-11-26 07:02:00.000000	Not Cancelled
10	Driver_10	140	2023-03-20 07:55:00.000000	Not Cancelled
10	Driver_10	141	2023-05-11 19:42:00.000000	Not Cancelled
10	Driver_10	142	2023-02-26 22:22:00.000000	Not Cancelled
10	Driver_10	143	2023-10-08 17:25:00.000000	Not Cancelled
10	Driver_10	144	2023-01-14 11:08:00.000000	Not Cancelled
10	Driver_10	145	2023-01-23 12:37:00.000000	Not Cancelled
10	Driver_10	146	2023-01-24 17:15:00.000000	Not Cancelled
10	Driver_10	147	2023-11-13 05:54:00.000000	Not Cancelled
10	Driver_10	148	2023-02-03 07:06:00.000000	Not Cancelled
10	Driver_10	149	2023-08-14 18:29:00.000000	Not Cancelled
10	Driver_10	150	2023-04-24 17:08:00.000000	Not Cancelled
10	Driver_10	151	2023-02-02 01:40:00.000000	Not Cancelled
10	Driver_10	152	2023-05-22 13:52:00.000000	Not Cancelled
10	Driver_10	153	2023-[illegible].000000	Not Cancelled
10	Driver_10	154	2023-[illegible].000000	Not Cancelled

55 rows

```sql
-- 3.1.5 Make the results easier to understand. Create a query with calculated/computed columns that would make sense to someone who does not work with SQL.
--   Use CASE where appropriate to create plain-language categories.
SELECT  trip_id,requested_at,  352 ms
    CASE EXTRACT(MONTH FROM requested_at)
        WHEN 1 THEN 'Jan'
        WHEN 2 THEN 'Feb'
        WHEN 3 THEN 'Mar'
        WHEN 4 THEN 'Apr'
        WHEN 5 THEN 'May'
        WHEN 6 THEN 'Jun'
        WHEN 7 THEN 'Jul'
        WHEN 8 THEN 'Aug'
        WHEN 9 THEN 'Sep'
        WHEN 10 THEN 'Oct'
        WHEN 11 THEN 'Nov'
        WHEN 12 THEN 'Dec'
        ELSE 'Unknown'
    END AS requested_month
FROM trips;
```

;

Output | Result 63

	trip_id	requested_at	requested_month
1	1	2023-06-06 21:55:00.000000	Jun
2	2	2023-06-09 08:54:00.000000	Jun
3	3	2023-08-19 19:03:00.000000	Aug
4	4	2023-01-01 15:48:00.000000	Jan
5	5	2023-06-27 21:17:00.000000	Jun
6	6	2023-02-08 00:47:00.000000	Feb
7	7	2023-02-23 10:07:00.000000	Feb
8	8	2023-01-16 21:46:00.000000	Jan
9	9	2023-03-26 10:22:00.000000	Mar
10	10	2023-06-22 22:30:00.000000	Jun
11	11	2023-11-14 09:05:00.000000	Nov
12	12	2023-01-20 08:07:00.000000	Jan
13	13	2023-04-12 22:27:00.000000	Apr

200 rows

Task 3.2: Find the bug

The dashboard query contains two SQL problems. Your goal is to reproduce each problem, show what goes wrong, and then fix it.

Part A: The rows that vanished

SQL uses three-valued logic: TRUE, FALSE, and UNKNOWN. A comparison against NULL yields UNKNOWN, and a WHERE clause keeps only rows where the predicate is TRUE. A filter that looks exhaustive can therefore discard rows silently, without an error and without warning anyone.

Reproduce it:

1. Pick a nullable column on your event or producer table. If the dataset has no NULLs in it, insert a few deliberately and say that you did.
2. Write a query with a negation or inequality predicate on that column — for example WHERE status != 'completed' — and record the row count it returns.
3. Write a second query counting the rows the first one silently omitted, and show that the two counts do not sum to the table total.
4. Repair the first query using an explicit IS NULL condition or COALESCE, so it returns what you intended.

Part B: The alias that did not exist yet

SQL is written FROM, WHERE, SELECT, ORDER BY, but it is not evaluated in that order.

1. Write a query defining an alias in SELECT and referencing that alias in WHERE. Capture the error message verbatim.
2. Produce two correct rewrites: one repeating the expression in WHERE, one wrapping the query in a CTE or subquery so the alias becomes a real column.

```sql
-- Task 3.2: Find the bug - Part A: The rows that vanished
-- Write a query with a negation or inequality predicate on that column — for example WHERE status != 'completed' — and record the row count it returns.
-- 3.2.A.2 Count of no shows
SELECT count(*) AS rider_no_show  331 ms
FROM trips
WHERE cancellation_reason != 'Rider no show';
```

Output | rider_no_show:bigint

	rider_no_show
1	22

```sql
-- Task 3.2: Find the bug - Part A: The rows that vanished
-- Write a second query counting the rows the first one silently omitted, and show that the two counts do not sum to the table total.
-- Count Nulls
SELECT  COUNT(*) AS Omitted_count  339 ms
FROM trips
WHERE cancellation_reason is NULL;
```

Output | Omitted_count:bigint

	omitted_count
1	166

```sql
-- Task 3.2: Find the bug - Part A: The rows that vanished
-- Repair the first query using an explicit IS NULL condition or COALESCE, so it returns what you intended.
-- Counting the nulls with the no shows
SELECT COUNT(*) cancellation_reason  335 ms
FROM trips
WHERE  cancellation_reason != 'Rider no show'
OR cancellation_reason is null;
```

Output | cancellation_reason:bigint

	cancellation_reason
1	188

```sql
-- Task 3.2 Part A: the two row counts and the table total together, so a reader can see they do not sum. Then the repaired query returning the full count.
WITH counts AS (  334 ms
    SELECT
        (SELECT COUNT(*)
         FROM trips
         WHERE cancellation_reason != 'Rider no show') AS not_rider_no_show,

        (SELECT COUNT(*)
         FROM trips
         WHERE cancellation_reason IS NULL) AS null_count,

        (SELECT COUNT(*)
         FROM trips
         WHERE cancellation_reason != 'Rider no show'
            OR cancellation_reason IS NULL) AS repaired_count,

        (SELECT COUNT(*)
         FROM trips) AS total_count
)
SELECT
    not_rider_no_show,
    null_count,
    repaired_count,
    total_count
FROM counts;
```

Services

Services > Database > postgres@localhost > q3_2.sql

Tx, Output | Result 82

	not_rider_no_…	null_count	repaired_count	total_count
1	22	166	188	200

```sql

-- Task 3.2:  Part B: The alias that did not exist yet
-- Write a query defining an alias in SELECT and referencing that alias in WHERE. Capture the error message verbatim.
SELECT  trip_id,requested_at,  19 ms
    CASE EXTRACT(MONTH FROM requested_at)
        WHEN 1 THEN 'Jan'
        WHEN 2 THEN 'Feb'
        WHEN 3 THEN 'Mar'
        WHEN 4 THEN 'Apr'
        WHEN 5 THEN 'May'
        WHEN 6 THEN 'Jun'
        WHEN 7 THEN 'Jul'
        WHEN 8 THEN 'Aug'
        WHEN 9 THEN 'Sep'
        WHEN 10 THEN 'Oct'
        WHEN 11 THEN 'Nov'
        WHEN 12 THEN 'Dec'
        ELSE 'Unknown'
    END AS requested_month
FROM trips
WHERE requested_month = 'Jun';

```

[42703] ERROR: column "requested_month" does not exist
  Position: 455

```sql
-- Task 3.2:  Part B: The alias that did not exist yet
-- Produce two correct rewrites: one repeating the expression in WHERE
-- Repeat the expression in WHERE instead of using the SELECT alias.
SELECT trip_id, requested_at,  343 ms
    CASE EXTRACT(MONTH FROM requested_at)
        WHEN 1 THEN 'Jan'
        WHEN 2 THEN 'Feb'
        WHEN 3 THEN 'Mar'
        WHEN 4 THEN 'Apr'
        WHEN 5 THEN 'May'
        WHEN 6 THEN 'Jun'
        WHEN 7 THEN 'Jul'
        WHEN 8 THEN 'Aug'
        WHEN 9 THEN 'Sep'
        WHEN 10 THEN 'Oct'
        WHEN 11 THEN 'Nov'
        WHEN 12 THEN 'Dec'
        ELSE 'Unknown'
    END AS requested_month
FROM trips
WHERE CASE EXTRACT(MONTH FROM requested_at)
        WHEN 1 THEN 'Jan'
        WHEN 2 THEN 'Feb'
        WHEN 3 THEN 'Mar'
        WHEN 4 THEN 'Apr'
        WHEN 5 THEN 'May'
        WHEN 6 THEN 'Jun'
        WHEN 7 THEN 'Jul'
        WHEN 8 THEN 'Aug'
        WHEN 9 THEN 'Sep'
        WHEN 10 THEN 'Oct'
        WHEN 11 THEN 'Nov'
        WHEN 12 THEN 'Dec'
        ELSE 'Unknown'
    END = 'Jan';
```

Services > Database > postgres@localhost > q3_2.sql

Output | Result 63

CSV

	trip_id	requested_at	requested_month
1	4	2023-01-01 15:48:00.000000	Jan
2	8	2023-01-16 21:46:00.000000	Jan
3	12	2023-01-20 08:07:00.000000	Jan
4	14	2023-01-27 17:06:00.000000	Jan
5	19	2023-01-20 11:49:00.000000	Jan
6	48	2023-01-25 21:19:00.000000	Jan
7	55	2023-01-28 12:37:00.000000	Jan
8	56	2023-01-08 08:12:00.000000	Jan
9	75	2023-01-28 20:59:00.000000	Jan
10	82	2023-01-13 17:20:00.000000	Jan
11	93	2023-01-27 23:39:00.000000	Jan
12	137	2023-01-05 14:18:00.000000	Jan
13	144	2023-01-14 10:57:00.000000	Jan
14	145	2023-01-23 11:44:00.000000	Jan
15	146	2023-01-24 17:09:00.000000	Jan
16	154	2023-01-23 22:51:00.000000	Jan
17	169	2023-01-08 14:28:00.000000	Jan
18	183	2023-01-10 21:57:00.000000	Jan
19	188	2023-01-07 08:04:00.000000	Jan
20	190	2023-01-08 02:11:00.000000	Jan
21	192	2023-01-10 10:35:00.000000	Jan

21 rows

```sql
-- Task 3.2:  Part B: The alias that did not exist yet
-- Produce two correct rewrites: one wrapping the query in a CTE or subquery so the alias becomes a real column.
WITH trip_months AS (  343 ms
    SELECT trip_id, requested_at,
        CASE EXTRACT(MONTH FROM requested_at)
            WHEN 1 THEN 'Jan'
            WHEN 2 THEN 'Feb'
            WHEN 3 THEN 'Mar'
            WHEN 4 THEN 'Apr'
            WHEN 5 THEN 'May'
            WHEN 6 THEN 'Jun'
            WHEN 7 THEN 'Jul'
            WHEN 8 THEN 'Aug'
            WHEN 9 THEN 'Sep'
            WHEN 10 THEN 'Oct'
            WHEN 11 THEN 'Nov'
            WHEN 12 THEN 'Dec'
            ELSE 'Unknown'
        END AS requested_month
    FROM trips
)
SELECT trip_id, requested_at, requested_month
FROM trip_months
WHERE requested_month = 'Jan';
```

Services

Services > Database > postgres@localhost > q3_2.sql

Output | Result 64

	trip_id	requested_at	requested_m
1	4	2023-01-01 15:48:00.000000	Jan
2	8	2023-01-16 21:46:00.000000	Jan
3	12	2023-01-20 08:07:00.000000	Jan
4	14	2023-01-27 17:06:00.000000	Jan
5	19	2023-01-20 11:49:00.000000	Jan
6	48	2023-01-25 21:19:00.000000	Jan
7	55	2023-01-28 12:37:00.000000	Jan
8	56	2023-01-08 08:12:00.000000	Jan
9	75	2023-01-28 20:59:00.000000	Jan
10	82	2023-01-13 17:20:00.000000	Jan
11	93	2023-01-27 23:39:00.000000	Jan
12	137	2023-01-05 14:18:00.000000	Jan
13	144	2023-01-14 10:57:00.000000	Jan
14	145	2023-01-23 11:44:00.000000	Jan
15	146	2023-01-24 17:09:00.000000	Jan
16	154	2023-01-23 22:51:00.000000	Jan
17	169	2023-01-08 14:28:00.000000	Jan
18	183	2023-01-10 21:57:00.000000	Jan
19	188	2023-01-07 08:04:00.000000	Jan
20	190	2023-01-08 02:11:00.000000	Jan
21	192	2023-01-10 10:35:00.000	

21 rows

```sql
-- Task 3.2:  Part B: The alias that did not exist yet
-- Produce two correct rewrites: one wrapping the query in a CTE or subquery so the alias becomes a real column.
-- subquery
SELECT trip_id, requested_at, requested_month
FROM (
    SELECT trip_id, requested_at,
        CASE EXTRACT(MONTH FROM requested_at)
            WHEN 1 THEN 'Jan'
            WHEN 2 THEN 'Feb'
            WHEN 3 THEN 'Mar'
            WHEN 4 THEN 'Apr'
            WHEN 5 THEN 'May'
            WHEN 6 THEN 'Jun'
            WHEN 7 THEN 'Jul'
            WHEN 8 THEN 'Aug'
            WHEN 9 THEN 'Sep'
            WHEN 10 THEN 'Oct'
            WHEN 11 THEN 'Nov'
            WHEN 12 THEN 'Dec'
            ELSE 'Unknown'
        END AS requested_month
    FROM trips
) AS trip_months
WHERE requested_month = 'Jan';
```

CSV

	trip_id	requested_at	requested_month
1	4	2023-01-01 15:48:00.000000	Jan
2	8	2023-01-16 21:46:00.000000	Jan
3	12	2023-01-20 08:07:00.000000	Jan
4	14	2023-01-27 17:06:00.000000	Jan
5	19	2023-01-20 11:49:00.000000	Jan
6	48	2023-01-25 21:19:00.000000	Jan
7	55	2023-01-28 12:37:00.000000	Jan
8	56	2023-01-08 08:12:00.000000	Jan
9	75	2023-01-28 20:59:00.000000	Jan
10	82	2023-01-13 17:20:00.000000	Jan
11	93	2023-01-27 23:39:00.000000	Jan
12	137	2023-01-05 14:18:00.000000	Jan
13	144	2023-01-14 10:57:00.000000	Jan
14	145	2023-01-23 11:44:00.000000	Jan
15	146	2023-01-24 17:09:00.000000	Jan
16	154	2023-01-23 22:51:00.000000	Jan
17	169	2023-01-08 14:28:00.000000	Jan
18	183	2023-01-10 21:57:00.000000	Jan
19	188	2023-01-07 08:04:00.000000	Jan
20	190	2023-01-08 02:11:00.000000	Jan
21	192	2023-01-10 10:35:00.000000	Jan

Task 3.3: Hand the team a corrected query

You want to give the dashboard team a query they can trust. The same question can be asked in several ways.

What to do

Answer a two-step question about your data — for example, which actors were involved in the disputed events — and express it three ways.

1. First as a subquery using IN or NOT IN.
2. Then as a Common Table Expression using WITH.
3. Then a third way, distinct from both — a different subquery form, a CASE-based construction, or a set-based approach.
4. Demonstrate that all three return identical result sets. Matching row counts is not enough: show that the rows themselves match.
5. Name one input condition under which the three would not be equivalent. NULLs, duplicates, and empty result sets are the usual places equivalence breaks.

```sql
-- Task 3.3: Hand the team a corrected query
-- Which riders were involved in rider no-show trips?
-- First as a subquery using IN or NOT IN.
SELECT rider_id, rider_name  350 ms
FROM riders
WHERE rider_id IN (
    SELECT rider_id
    FROM trips
    WHERE cancellation_reason = 'Rider no show'
)
ORDER BY rider_id;
```

Tx: Auto | DDL

	rider_id	rider_name
1	1	Rider_01
2	4	Rider_04
3	6	Rider_06
4	7	Rider_07
5	15	Rider_15
6	17	Rider_17
7	19	Rider_19
8	21	Rider_21
9	27	Rider_27
10	29	Rider_29

```sql
-- Task 3.3: Hand the team a corrected query
-- Which riders were involved in rider no-show trips?
-- common table expression
WITH no_show_riders AS (  340 ms
    SELECT rider_id
    FROM trips
    WHERE cancellation_reason = 'Rider no show'
)
SELECT rider_id, rider_name
FROM riders
WHERE rider_id IN (
    SELECT rider_id
    FROM no_show_riders
)
ORDER BY rider_id;
```

Tx: Auto | DDL

	rider_id	rider_name
1	1	Rider_01
2	4	Rider_04
3	6	Rider_06
4	7	Rider_07
5	15	Rider_15
6	17	Rider_17
7	19	Rider_19
8	21	Rider_21
9	27	Rider_27
10	29	Rider_29

```sql
-- Task 3.3: Hand the team a corrected query
-- Which riders were involved in rider no-show trips?
-- third way Join
SELECT DISTINCT r.rider_id, r.rider_name  339 ms
FROM riders r
JOIN trips t
    1<->1..n: ON r.rider_id = t.rider_id
WHERE t.cancellation_reason = 'Rider no show'
ORDER BY r.rider_id;
```

Tx: Auto | DDL

	rider_id	rider_name
1	1	Rider_01
2	4	Rider_04
3	6	Rider_06
4	7	Rider_07
5	15	Rider_15
6	17	Rider_17
7	19	Rider_19
8	21	Rider_21
9	27	Rider_27
10	29	Rider_29

```sql
-- Task 3.3: Hand the team a corrected query
-- Which riders were involved in rider no-show trips?
-- A fourth way would be to do it with EXISTS
SELECT rider_id, rider_name  354 ms
FROM riders r
WHERE EXISTS (
    SELECT 1
    FROM trips t
    WHERE t.rider_id = r.rider_id
      AND t.cancellation_reason = 'Rider no show'
)
ORDER BY r.rider_id;
```

	rider_id	rider_name
1	1	Rider_01
2	4	Rider_04
3	6	Rider_06
4	7	Rider_07
5	15	Rider_15
6	17	Rider_17
7	19	Rider_19
8	21	Rider_21
9	27	Rider_27
10	29	Rider_29

	rider_id	rider_name
1	1	Rider_01
2	4	Rider_04
3	6	Rider_06
4	7	Rider_07
5	15	Rider_15
6	17	Rider_17
7	19	Rider_19
8	21	Rider_21
9	27	Rider_27
10	29	Rider_29

	rider_id	rider_name
1	1	Rider_01
2	4	Rider_04
3	6	Rider_06
4	7	Rider_07
5	15	Rider_15
6	17	Rider_17
7	19	Rider_19
8	21	Rider_21
9	27	Rider_27
10	29	Rider_29

	rider_id	rider_name
1	1	Rider_01
2	4	Rider_04
3	6	Rider_06
4	7	Rider_07
5	15	Rider_15
6	17	Rider_17
7	19	Rider_19
8	21	Rider_21
9	27	Rider_27
10	29	Rider_29

```
-- 3.3.5 Duplicate rider IDs in matching trips would make the JOIN return duplicate riders if DISTINCT were not used.
```